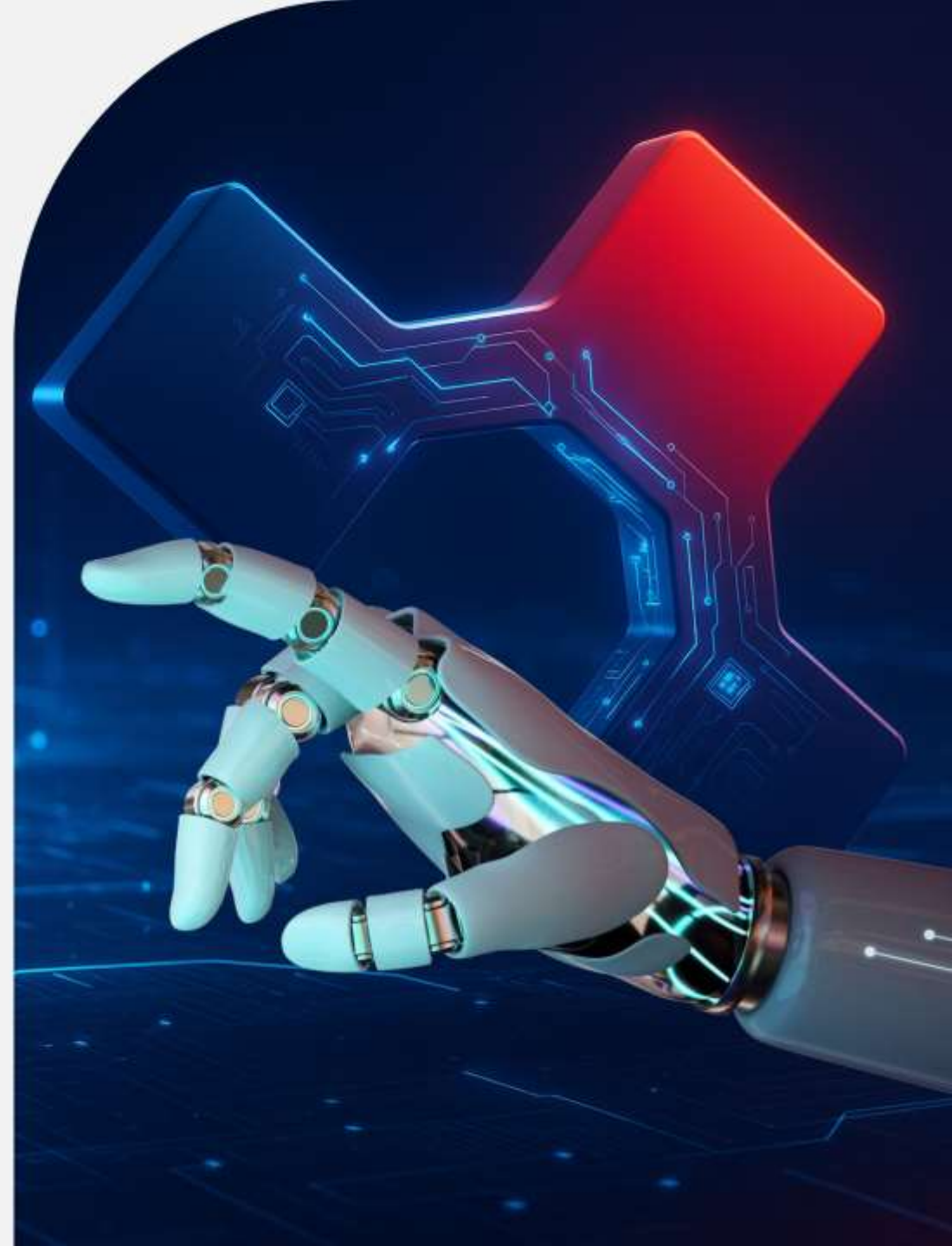
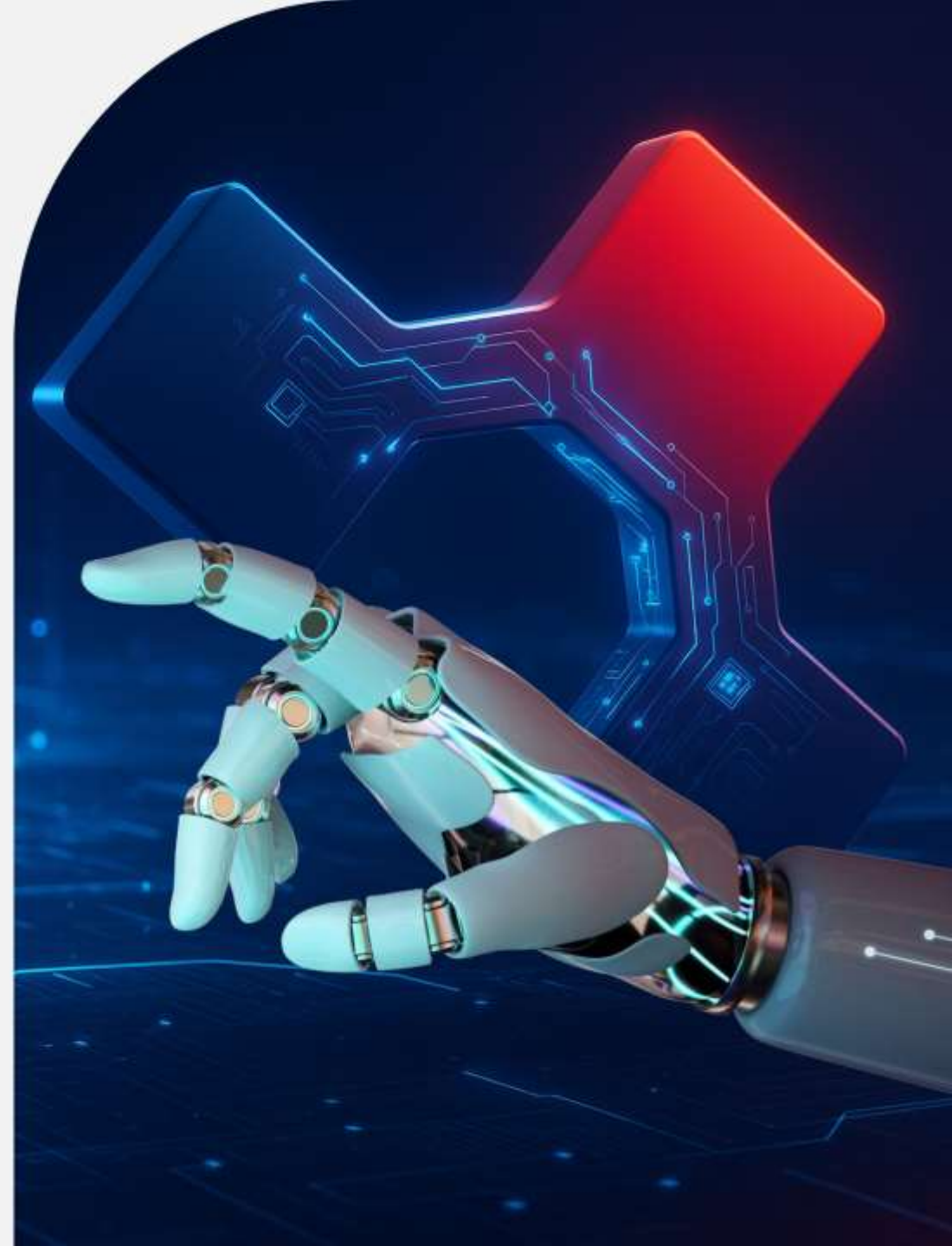


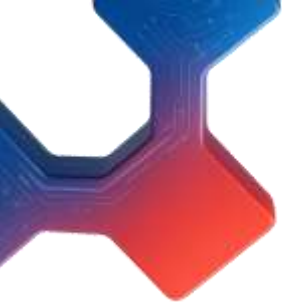


Núcleo de Capacitação em Inteligência Artificial



Aula 16





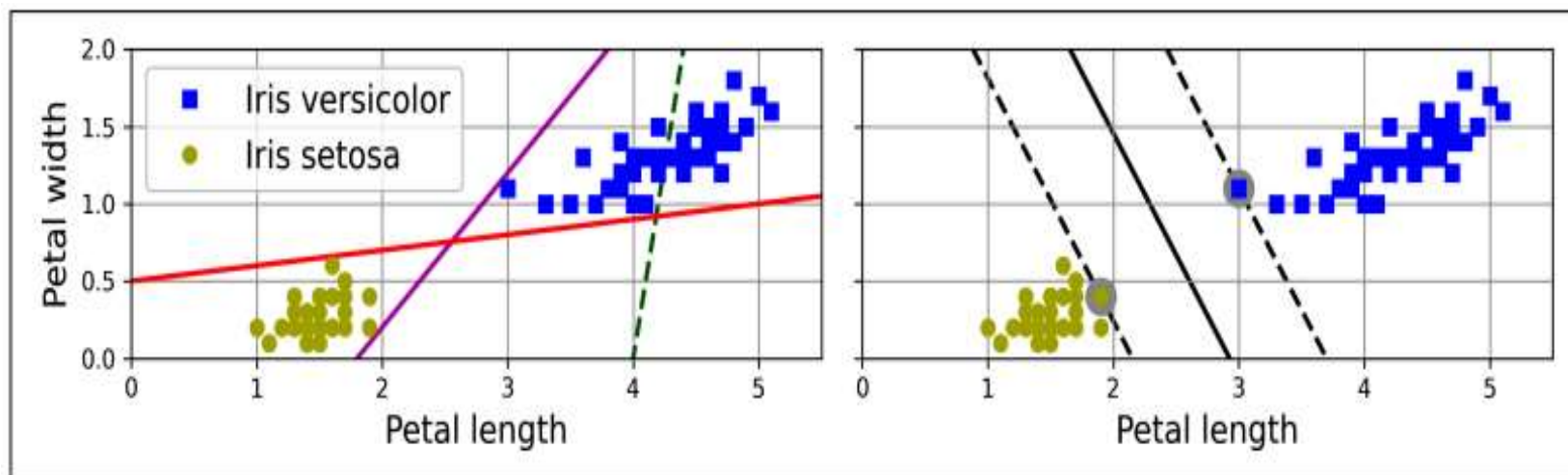
Máquinas de Vetores de Suporte

1. Introdução

- Uma máquina de vetores de suporte (SVM) é um modelo muito robusto e versátil de aprendizado de máquina, capaz de fazer classificações lineares ou não-lineares, de regressão e até mesmo detecção de outliers.
- As SVMs são particularmente adaptadas para a classificação de conjuntos de dados complexos pequenos ou de médio porte.

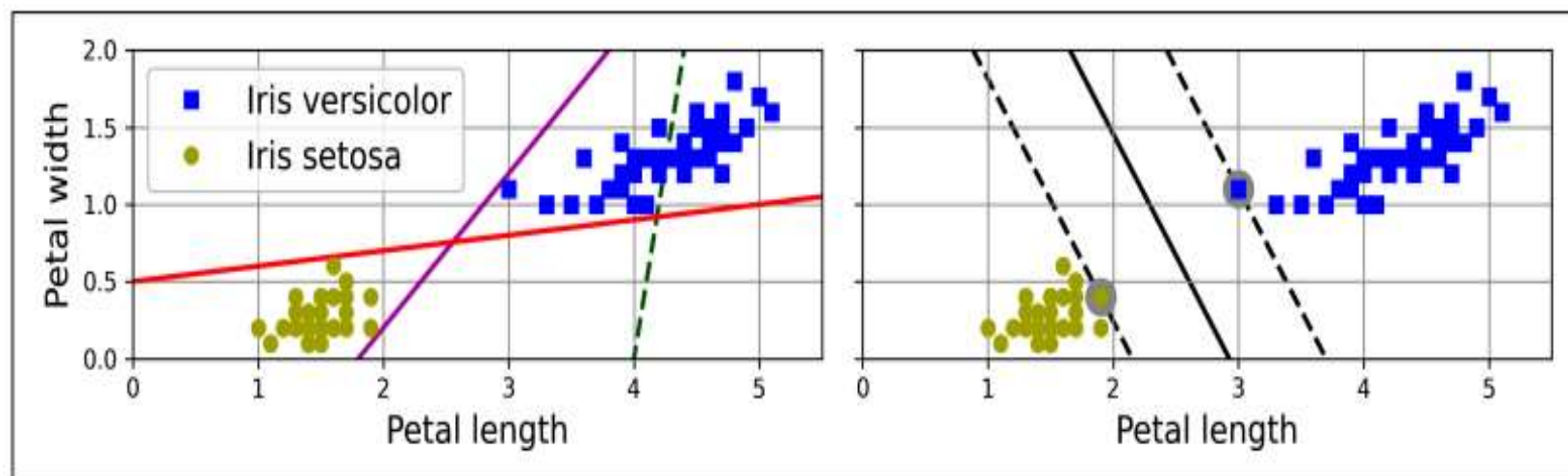
1. Classificação Linear da SVM

- O gráfico à esquerda mostra os limites de decisão de três possíveis classificadores lineares. O modelo cujo limite de decisão é representado pela linha tracejada é tão ruim que nem consegue separar corretamente as classes. Os outros dois modelos funcionam perfeitamente neste conjunto de treinamento, mas seus limites de decisão passam tão próximos das instâncias que provavelmente não terão um bom desempenho em novas instâncias.



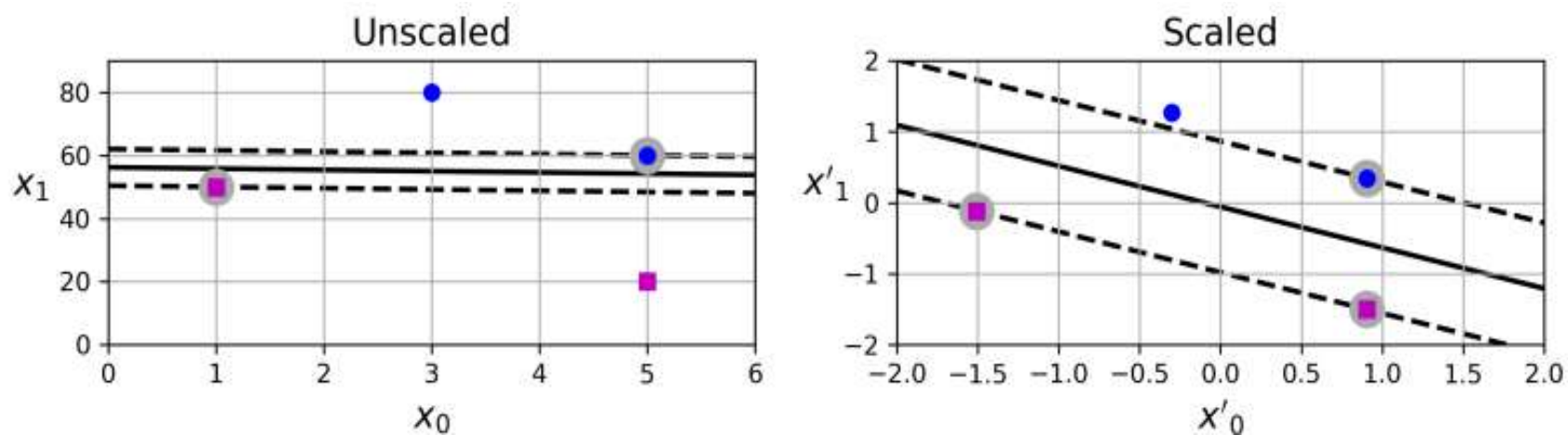
1. Classificação Linear da SVM

- Em contraste, a linha sólida no gráfico à direita representa o limite de decisão de um classificador SVM. Essa linha não apenas separa as duas classes, mas também mantém a maior distância possível em relação às instâncias de treinamento mais próximas. Você pode pensar em um classificador SVM como se ele estivesse ajustando a **rua mais larga possível** (representada pelas linhas tracejadas paralelas) entre as classes. Isso é chamado de **classificação por margem larga**.



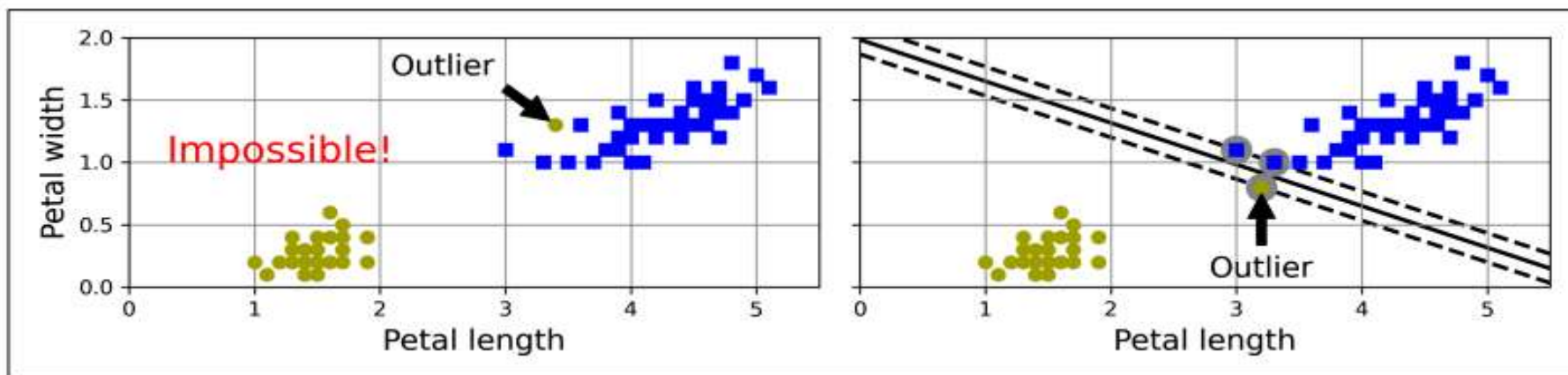
1. Classificação Linear da SVM

- As SVMs são sensíveis à escala das características, como mostrado na Figura 5-2. No gráfico à esquerda, a escala vertical é muito maior que a horizontal, fazendo com que a rua mais larga possível fique próxima da horizontal. Após o **escalonamento das características** (por exemplo, usando o **StandardScaler** do Scikit-Learn), o limite de decisão no gráfico à direita fica muito melhor.



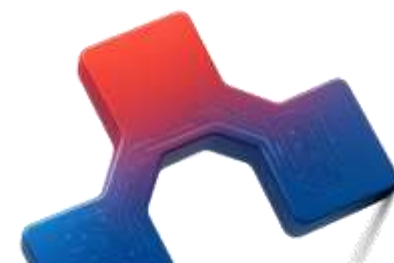
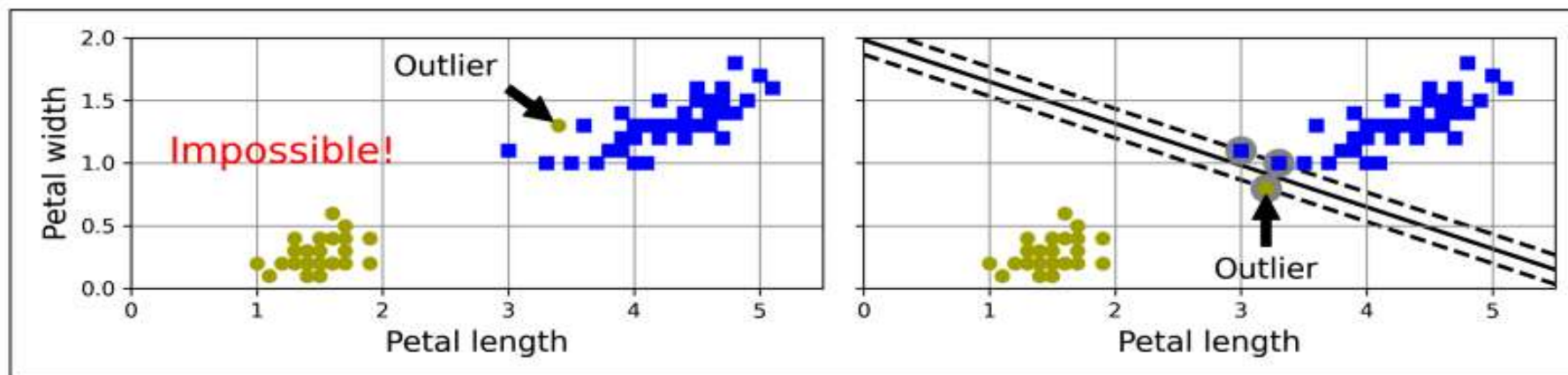
1. Classificação de Margem Suave

- Se exigirmos rigidamente que todas as instâncias fiquem fora da rua e no lado correto, isso é chamado de **classificação por margem rígida** (hard margin). Existem dois problemas principais com essa abordagem: primeiro, ela só funciona se os dados forem linearmente separáveis; segundo, é sensível a **outliers**. A Figura abaixo mostra o conjunto de dados **iris** com apenas um outlier adicional: à esquerda, é impossível encontrar uma margem rígida; à direita, o limite de decisão muda bastante em relação ao da **Figura primeiro slide** sem o outlier, e o modelo provavelmente não generalizará tão bem.



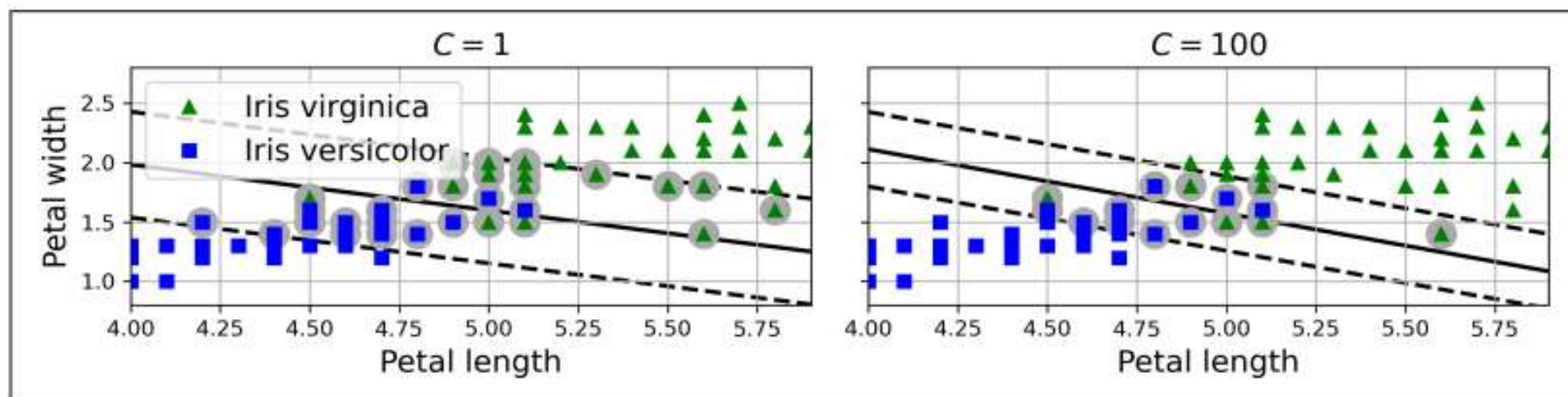
1. Classificação de Margem Suave

- Para evitar esses problemas, é necessário usar um modelo mais flexível. O objetivo é encontrar um **bom equilíbrio** entre manter a rua o mais larga possível e limitar as **violações da margem** (ou seja, instâncias que acabam no meio da rua ou até no lado errado). Isso é chamado de **classificação por margem suave** (soft margin).



1. Classificação de Margem Suave

Ao criar um modelo SVM usando o Scikit-Learn, é possível definir vários **hiperparâmetros**, incluindo o hiperparâmetro de **regularização C**. Valores baixos de C produzem o modelo à esquerda da Figura abaixo, enquanto valores altos geram o modelo à direita. Como é possível ver, reduzir C aumenta a largura da rua, mas também provoca mais **violações da margem**. Em outras palavras, um C menor faz com que mais instâncias suportem a rua, diminuindo o risco de **overfitting**. No entanto, se C for reduzido demais, o modelo pode **subajustar** os dados, como parece ser o caso aqui: o modelo com C=100 tende a generalizar melhor do que o com C=1.



1. Classificação de Margem Suave

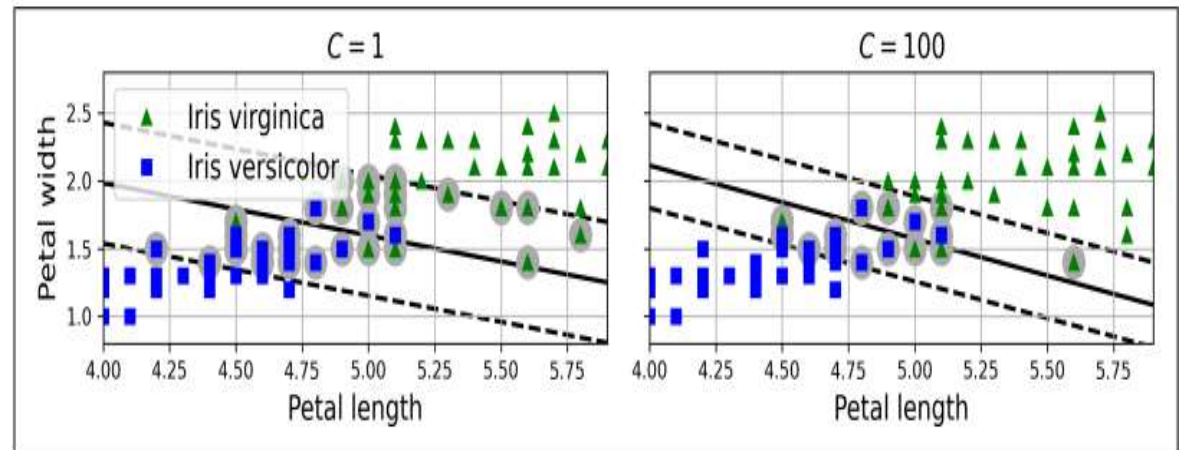
```
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2) # Iris virginica

svm_clf = make_pipeline(StandardScaler(),
                        LinearSVC(C=1, random_state=42))

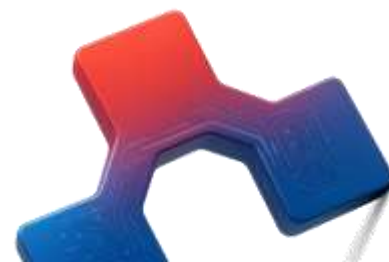
svm_clf.fit(X, y)

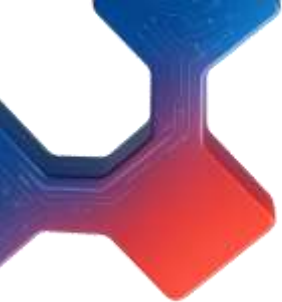
>>> X_new = [[5.5, 1.7], [5.0, 1.5]]
>>> svm_clf.predict(X_new)
array([ True, False])
```



A primeira planta é classificada como *Iris virginica*, enquanto a segunda não. Vejamos as pontuações que o SVM utilizou para fazer essas previsões. Elas medem a distância sinalizada entre cada instância e o limite de decisão.

```
>>> svm_clf.decision_function(X_new)
array([ 0.66163411, -0.22036063])
```





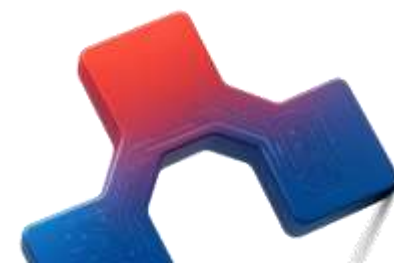
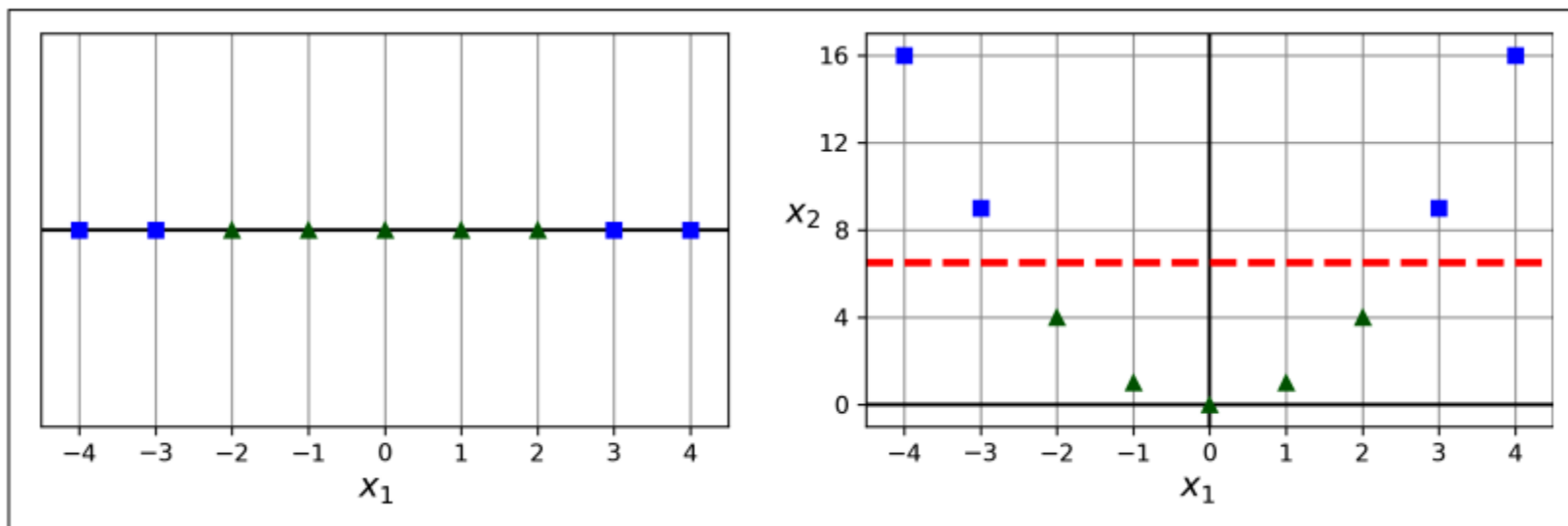
1. Classificação de Margem Suave

- Diferente da **LogisticRegression**, o **LinearSVC** não tem o método **predict_proba()** para calcular probabilidades das classes. Mas se você usar a classe **SVC** com **probability=True**, o modelo vai treinar uma etapa extra para transformar os resultados da SVM em probabilidades.
- Isso é feito usando **validação cruzada de 5 folds** para criar previsões fora da amostra e, em seguida, treinando um modelo de **LogisticRegression**, o que deixa o treinamento mais lento. Depois disso, os métodos **predict_proba()** e **predict_log_proba()** poderão ser usados.



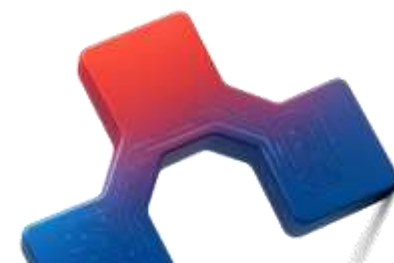
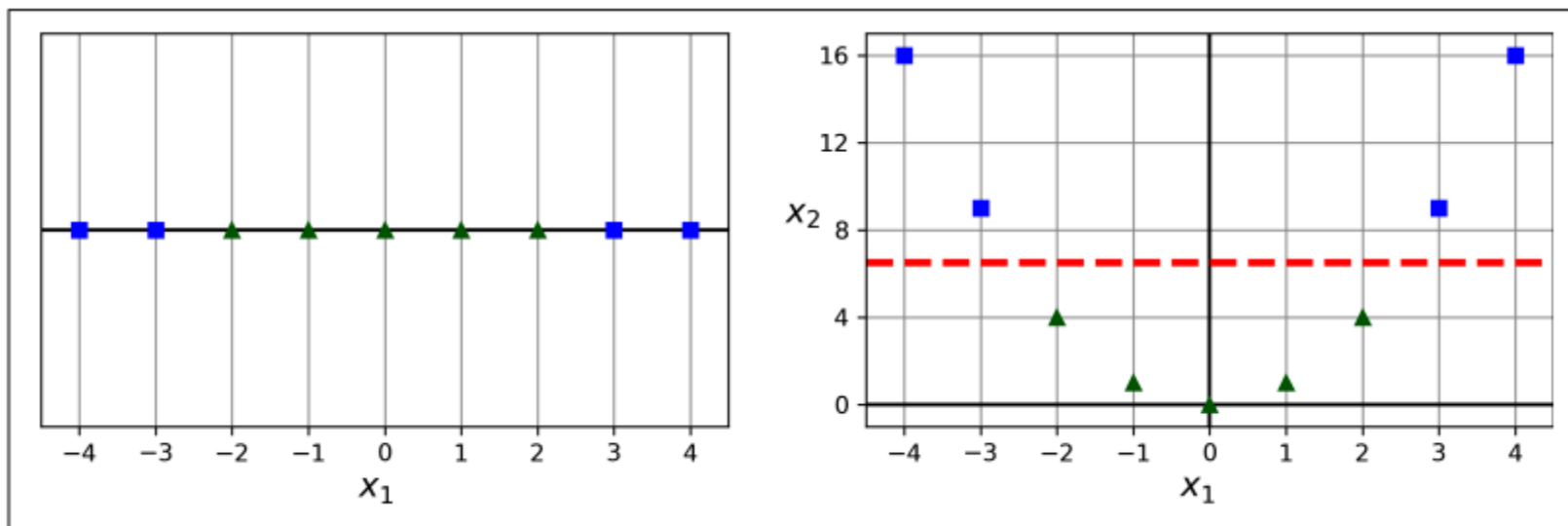
1. Classificação SVM não linear

- Embora os classificadores SVM lineares sejam eficientes e muitas vezes funcionem muito bem, muitos conjuntos de dados não são nem de longe linearmente separáveis. Uma forma de lidar com dados não lineares é **adicionar mais características**, como **features polinomiais**; em alguns casos, isso pode tornar o conjunto de dados linearmente separável.



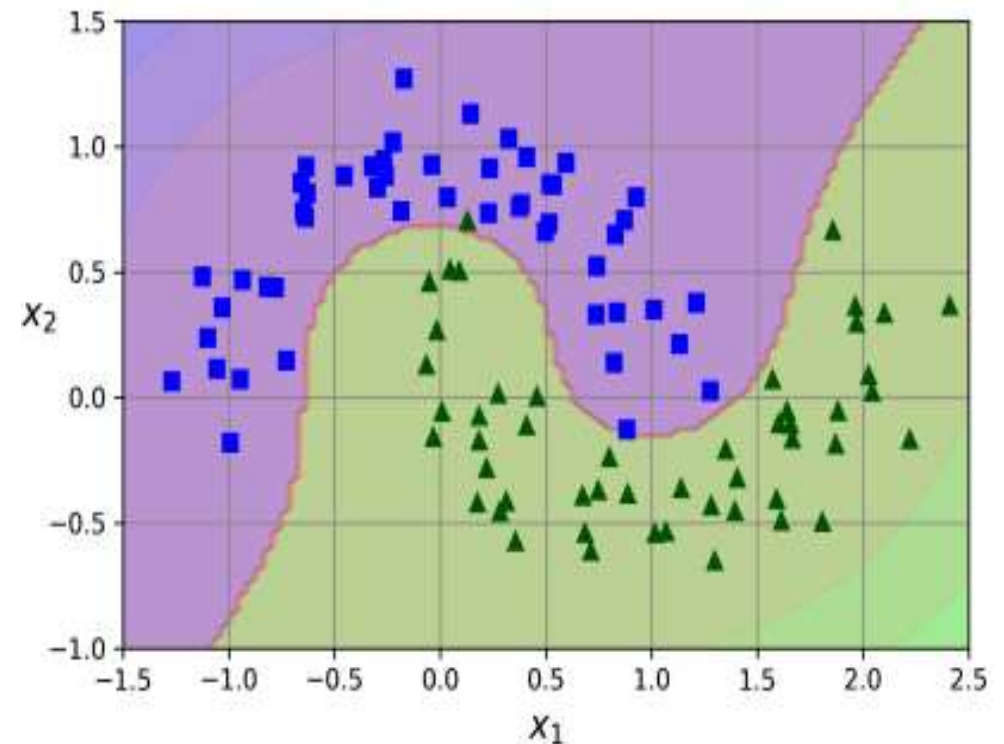
1. Classificação SVM não linear

- Por exemplo, no gráfico à esquerda da figura abaixo, temos um conjunto de dados simples com apenas uma característica, x_1 . Esse conjunto não é linearmente separável. Mas se adicionarmos uma segunda característica $x_2 = (x_1)^2$, o conjunto de dados resultante em 2D se torna perfeitamente linearmente separável.



1. Classificação SVM não linear

- Para implementar essa ideia no **Scikit-Learn**, você pode criar um **pipeline** que contenha um transformador **PolynomialFeatures** (como discutido em “Polynomial Regression”), seguido de um **StandardScaler** e um classificador **LinearSVC**.
- Vamos testar isso no **conjunto de dados moons**, um conjunto de exemplo para classificação binária em que os pontos formam duas luas entrelaçadas (veja a Figura ao lado). Esse conjunto de dados pode ser gerado usando a função **make_moons()**.

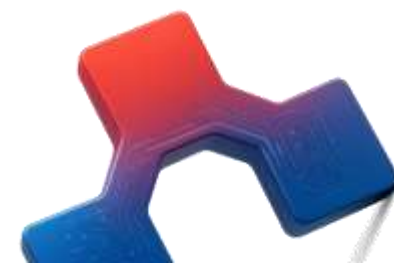
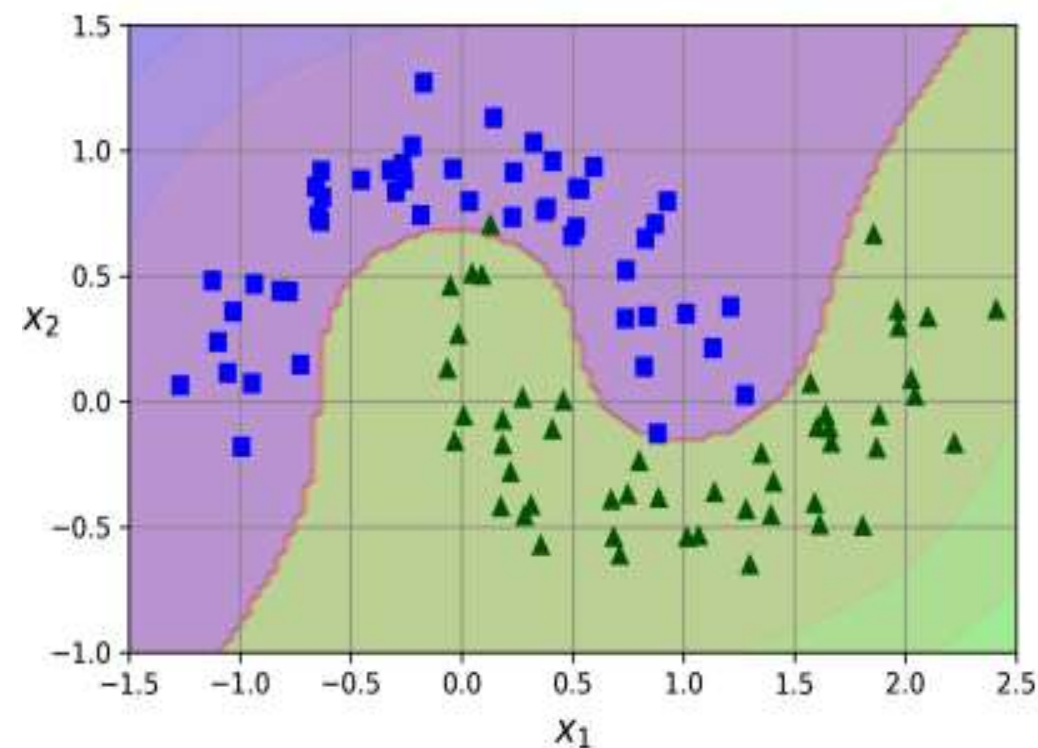


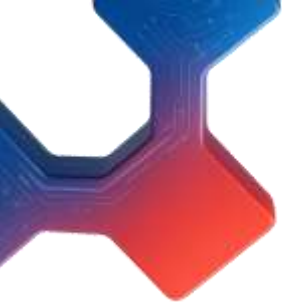
1. Classificação SVM não linear

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures

X, y = make_moons(n_samples=100, noise=0.15, random_state=42)

polynomial_svm_clf = make_pipeline(
    PolynomialFeatures(degree=3),
    StandardScaler(),
    LinearSVC(C=10, max_iter=10_000, random_state=42)
)
polynomial_svm_clf.fit(X, y)
```





1. Kernel polinomial

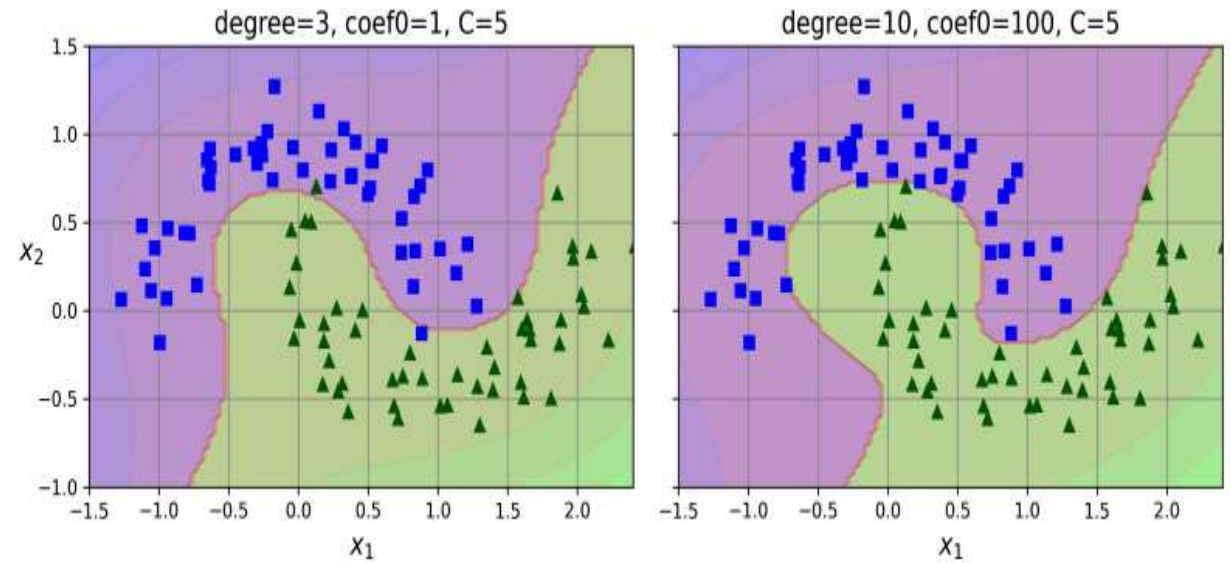
- Adicionar **features polinomiais** é simples de implementar e pode funcionar muito bem com diversos algoritmos de machine learning (não apenas SVMs). No entanto, com **grau polinomial baixo**, esse método não consegue lidar com conjuntos de dados muito complexos, e com **grau polinomial alto** ele cria um número enorme de features, deixando o modelo muito lento.
- Felizmente, ao usar SVMs, é possível aplicar uma técnica matemática quase milagrosa chamada **kernel trick** (explicada mais adiante neste capítulo). O kernel trick permite obter o mesmo resultado que se você tivesse adicionado muitas features polinomiais, mesmo com grau muito alto, **sem precisar adicioná-las de fato**. Isso evita a explosão combinatória do número de features. Essa técnica é implementada pela classe **SVC**. Vamos testá-la no conjunto de dados **moons**.

1. Kernel polinomial

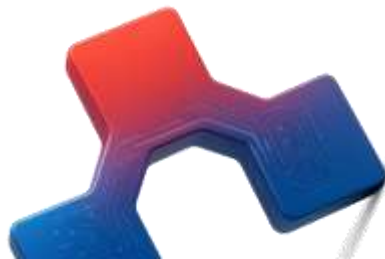
```
from sklearn.svm import SVC

poly_kernel_svm_clf = make_pipeline(StandardScaler(),
                                     SVC(kernel="poly", degree=3, coef0=1, C=5))

poly_kernel_svm_clf.fit(X, y)
```

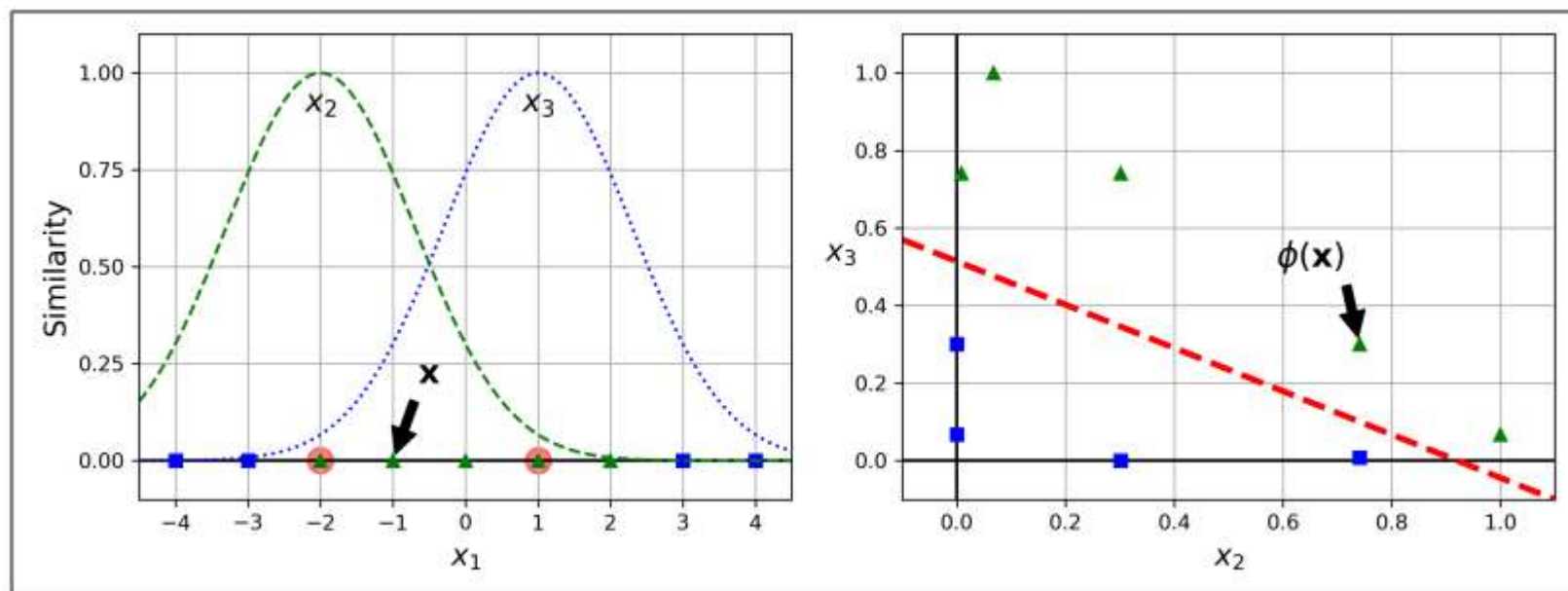


Este código treina um classificador SVM usando um **kernel polinomial de grau 3**, representado no gráfico à esquerda da acima. À direita está outro classificador SVM com **kernel polinomial de grau 10**.



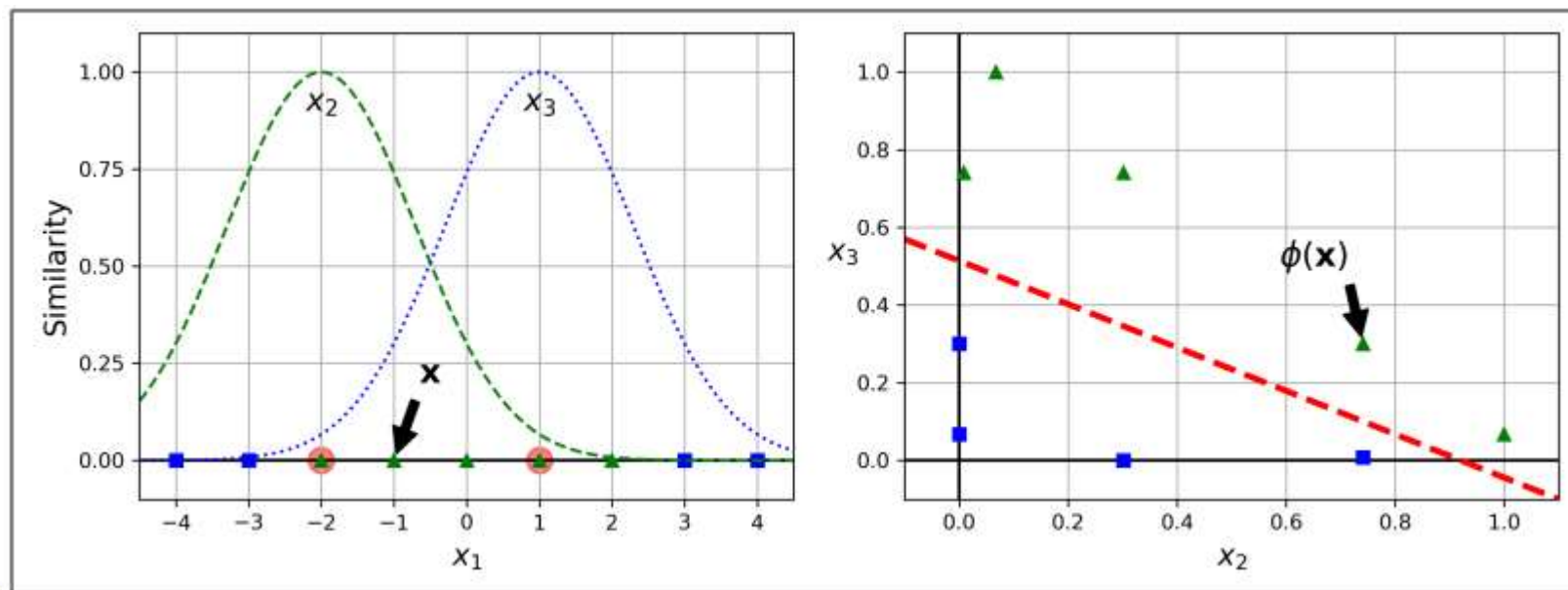
1. Características de similaridade

- Outra técnica para lidar com problemas não lineares é **adicionar features calculadas a partir de uma função de similaridade**, que mede o quanto cada instância se parece com um **marco específico**, como fizemos nas aulas anteriores ao adicionar as features de similaridade geográfica.



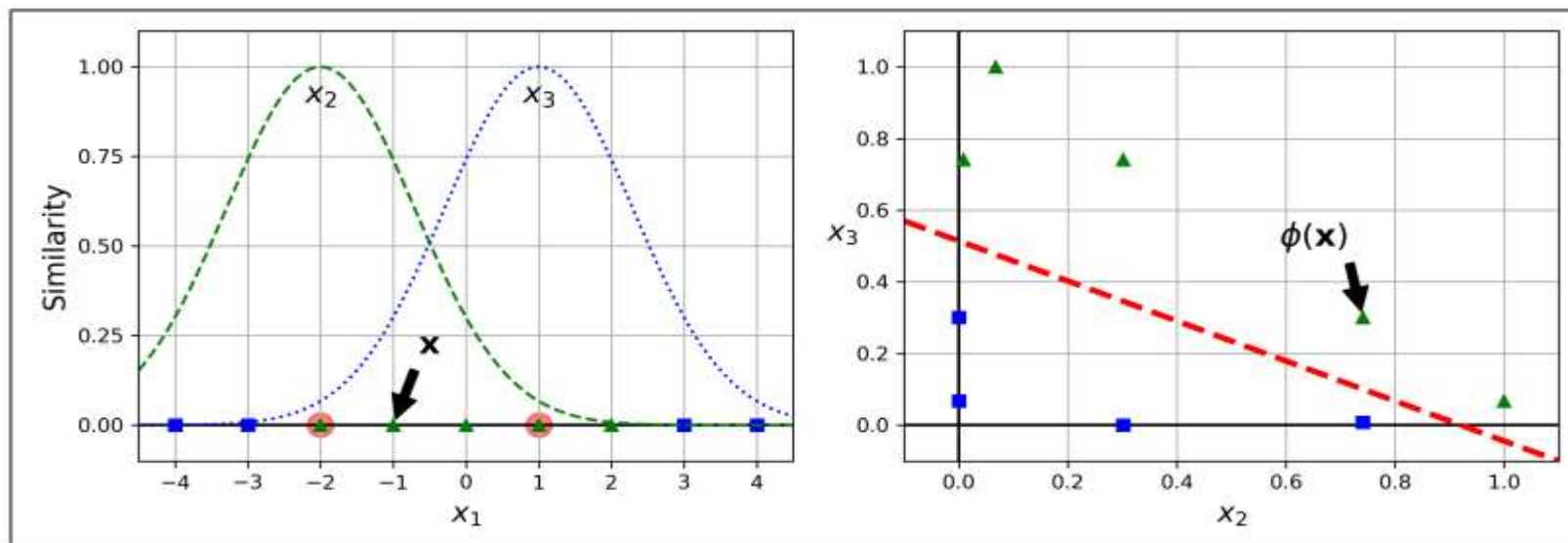
1. Características de similaridade

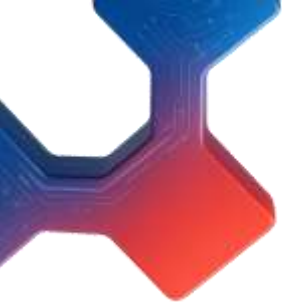
- Por exemplo, vamos pegar o conjunto de dados 1D usado anteriormente e adicionar **dois marcos** nele em $x_1 = -2$ e $x_1 = 1$ (veja o gráfico abaixo); Em seguida, definimos a **função de similaridade** como uma **RBF Gaussiana** com $\gamma = 0.3$. Essa é uma função em forma de sino, variando de 0 (muito distante do marco) a 1 (no próprio marco);



1. Características de similaridade

Agora estamos prontos para calcular as novas features. Por exemplo, vamos analisar a instância $x_1 = -1$: ela está a uma distância de 1 do primeiro marco e a 2 do segundo marco. Portanto, suas novas features são $x_2 = \exp(-0.3 \times 1^2) \approx 0.74$ e $x_3 = \exp(-0.3 \times 2^2) \approx 0.30$. O gráfico à direita da Figura 5-8 mostra o conjunto de dados transformado (descartando as features originais). Como você pode ver, ele agora é **linearmente separável**.





1. Características de similaridade

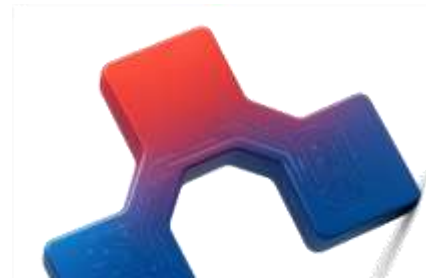
- Você pode estar se perguntando como selecionar os pontos de referência. A abordagem mais simples é criar um ponto de referência no local de cada instância no conjunto de dados. Isso cria muitas dimensões e, portanto, aumenta as chances de que o conjunto de treinamento transformado seja linearmente separável. A desvantagem é que um conjunto de treinamento com m instâncias e n recursos é transformado em um conjunto de treinamento com m instâncias e m recursos (assumindo que você descarte os recursos originais). Se o seu conjunto de treinamento for muito grande, você acabará com um número igualmente grande de recursos.



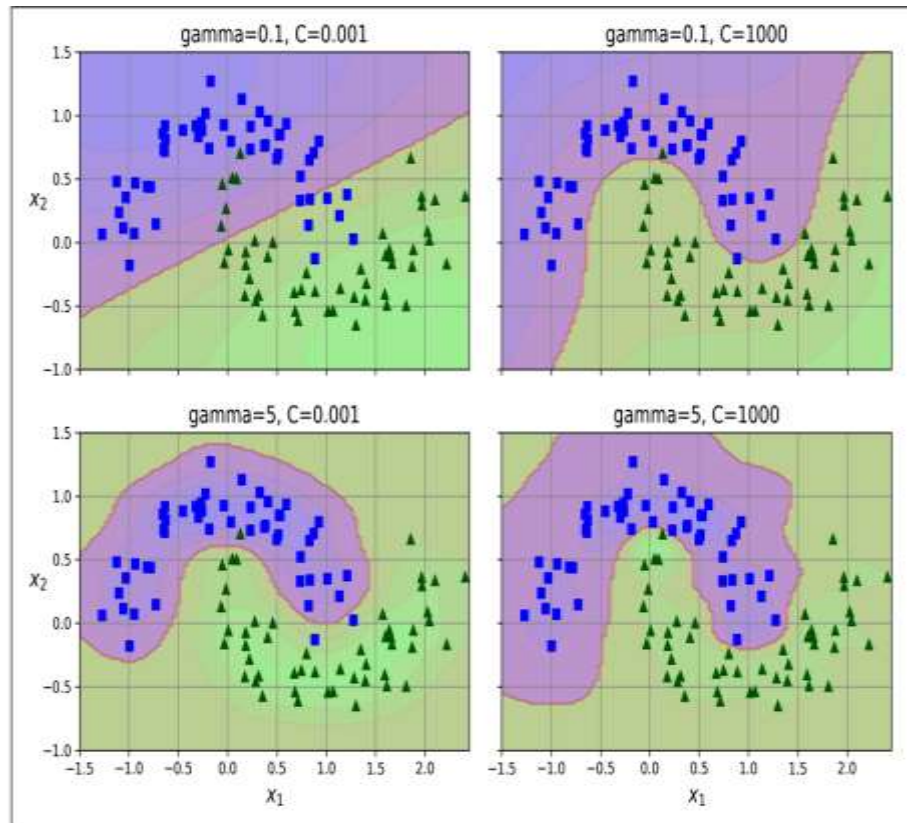
1. Kernel gaussiano RBF

- Assim como o método de **features polinomiais**, o método de **features de similaridade** pode ser útil com qualquer algoritmo de machine learning, mas pode ser **computacionalmente caro** calcular todas as features adicionais (especialmente em conjuntos de treinamento grandes).
- Mais uma vez, o **kernel trick** faz sua magia nas SVMs, permitindo obter um resultado semelhante ao de adicionar muitas features de similaridade, **sem precisar calculá-las de fato**. Vamos testar isso usando a classe **SVC** com o **kernel Gaussiano RBF**.

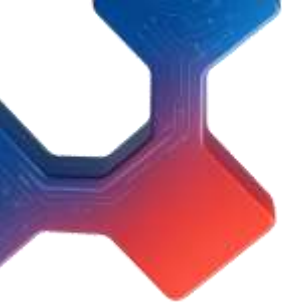
```
rbf_kernel_svm_clf = make_pipeline(StandardScaler(),  
                                   SVC(kernel="rbf", gamma=5, C=0.001))  
rbf_kernel_svm_clf.fit(X, y)
```



1. Kernel gaussiano RBF



- Este modelo é representado no canto inferior esquerdo da Figura 5-9. Os outros gráficos mostram modelos treinados com diferentes valores dos hiperparâmetros **gamma** (γ) e **C**.
- Aumentar γ torna a curva em forma de sino mais estreita (veja os gráficos à esquerda da lado). Como resultado, o **alcance de influência** de cada instância é menor, e o limite de decisão se torna mais irregular, contornando as instâncias individuais.
- Por outro lado, um valor pequeno de γ deixa a curva mais larga: as instâncias têm um alcance maior de influência e o limite de decisão fica mais **suave**. Assim, γ funciona como um



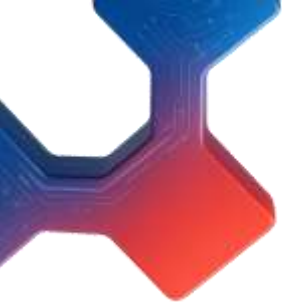
1. Kernel gaussiano RBF

- Existem outros **kernels**, mas são usados com muito menos frequência. Alguns kernels são especializados para **estruturas de dados específicas**. Por exemplo, **string kernels** são usados às vezes para classificar documentos de texto ou sequências de DNA (como o **string subsequence kernel** ou kernels baseados na **distância de Levenshtein**).
- Com tantos **kernels** disponíveis, como decidir qual usar? Como regra geral, você deve sempre **tentar o kernel linear primeiro**. A classe **LinearSVC** é muito mais rápida do que **SVC(kernel="linear")**, especialmente se o conjunto de treinamento for muito grande. Se o conjunto de dados não for tão grande, também vale a pena testar SVMs com **kernels**, começando pelo **kernel Gaussiano RBF**, que frequentemente funciona muito bem. Depois, se houver tempo e poder de computação disponíveis, você pode experimentar alguns outros kernels usando **busca de hiperparâmetros**. E se existirem kernels especializados para a estrutura de dados do seu conjunto de treinamento, vale a pena testá-los também.

1. Complexidade Computacional

- A classe **LinearSVC** é baseada na biblioteca **liblinear**, que implementa um algoritmo otimizado para SVMs lineares. Ela não suporta o **kernel trick**, mas sua escalabilidade é quase linear com o número de instâncias de treinamento e de features, com complexidade de treinamento aproximadamente $O(m \times n)$. O algoritmo demora mais se você exigir **alta precisão**, controlada pelo hiperparâmetro de tolerância ϵ (chamado **tol** no Scikit-Learn). Na maioria das tarefas de classificação, a tolerância padrão é suficiente.
- A classe **SVC** é baseada na biblioteca **libsvm**, que implementa um algoritmo que suporta o **kernel trick**. Sua complexidade de treinamento geralmente varia entre $O(m^2 \times n)$ e $O(m^3 \times n)$. Isso significa que ele se torna muito lento quando o número de instâncias de treinamento é grande (por exemplo, centenas de milhares), sendo mais adequado para conjuntos de dados não lineares de pequeno ou médio porte. O SVC escala bem com o número de features, especialmente quando são **sparsas** (ou seja, quando cada instância possui poucas features não nulas), aproximando-se da complexidade proporcional à média de features não nulas por instância.





1. Complexidade Computacional

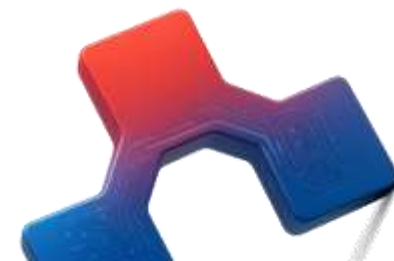
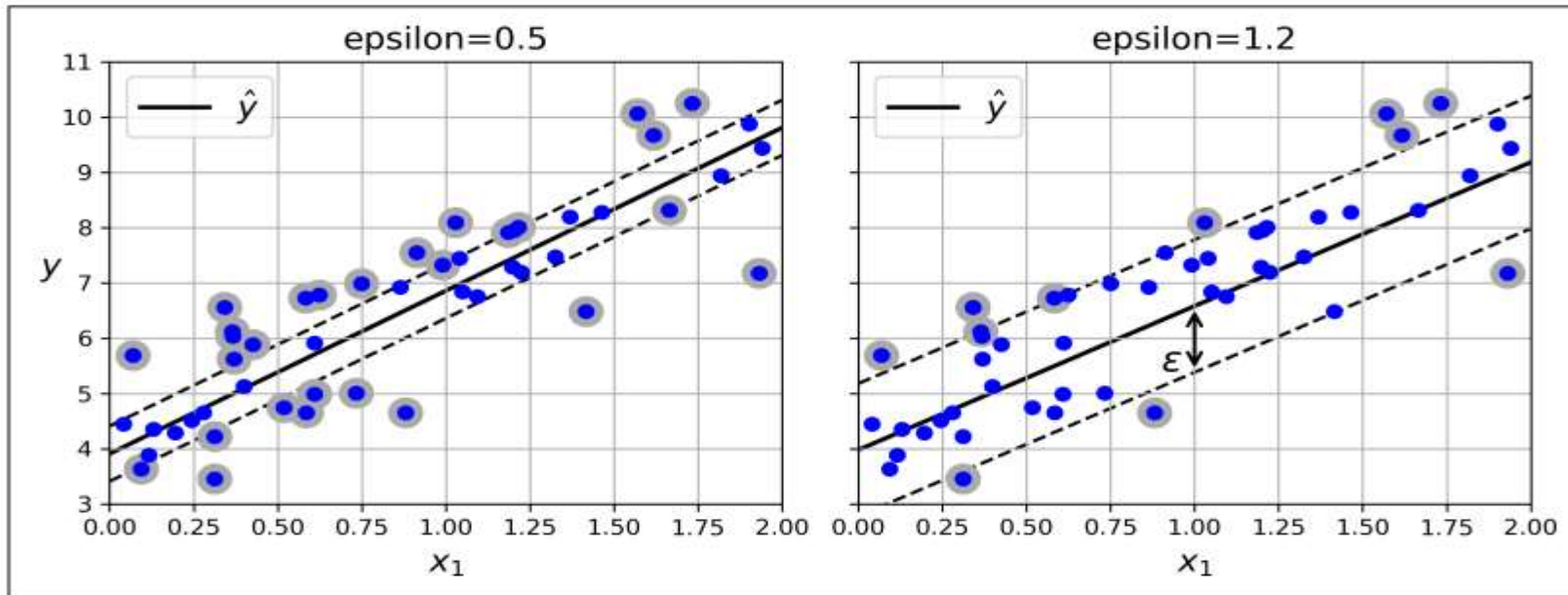
- A classe **SGDClassifier** também realiza classificação de **margem ampla** por padrão, e seus hiperparâmetros – especialmente os de regularização (α e **penalty**) e **learning_rate** – podem ser ajustados para obter resultados semelhantes aos do SVM linear. Ela utiliza **gradiente descendente estocástico**, permitindo **aprendizado incremental** e baixo uso de memória, útil para treinar modelos em grandes conjuntos de dados que não cabem na RAM (ou seja, **out-of-core learning**). Além disso, sua escalabilidade é excelente, com complexidade computacional $O(m \times n)$.

Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
LinearSVC	$O(m \times n)$	No	Yes	No
SVC	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes
SGDClassifier	$O(m \times n)$	Yes	Yes	No



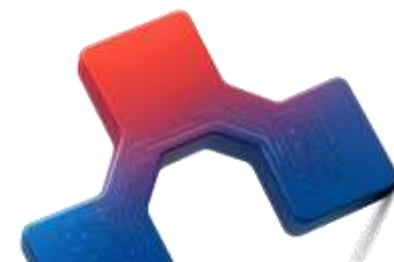
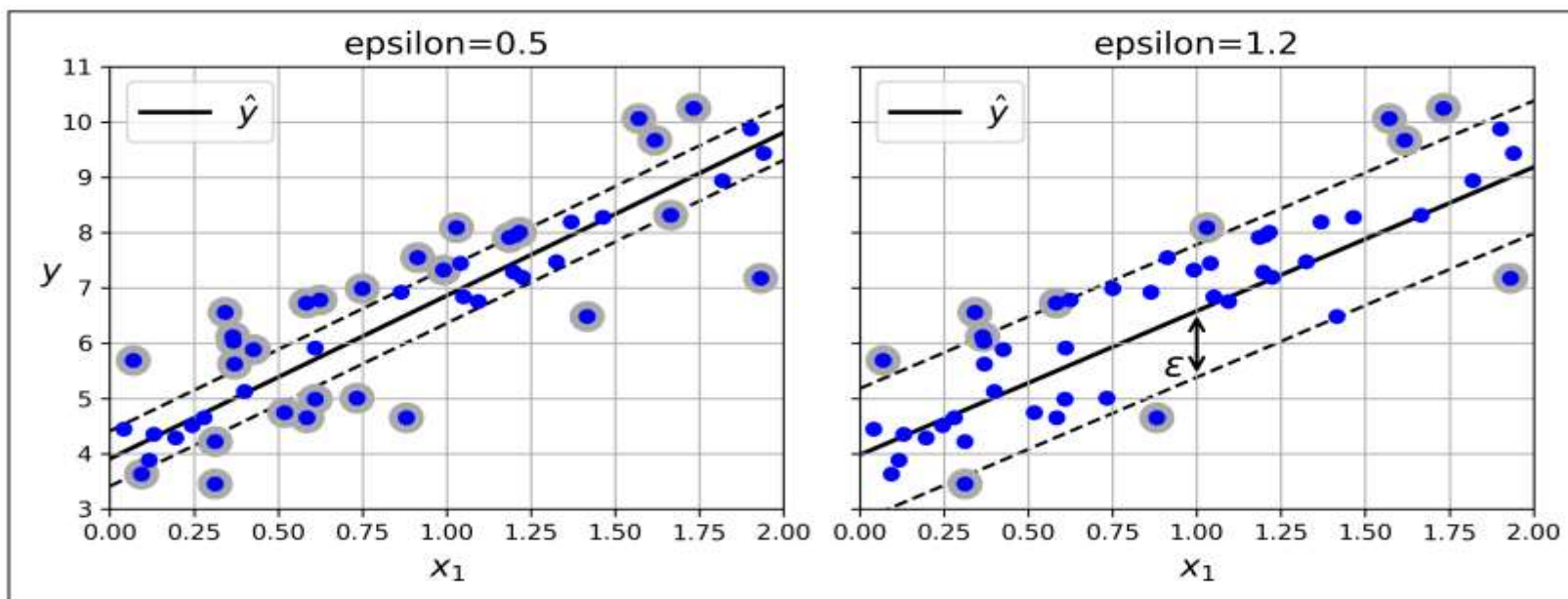
2. Regressão SVM

- Para usar SVMs para regressão em vez de classificação, o truque é ajustar o objetivo: em vez de tentar encaixar a maior “rua” possível entre duas classes, a regressão SVM procura ajustar o maior número de instâncias dentro dessa rua, permitindo algumas violações de margem (ou seja, instâncias fora da rua). A largura da rua é controlada por um hiperparâmetro, ϵ . A Figura abaixo mostra dois modelos lineares de regressão SVM treinados em alguns dados lineares: um com uma margem pequena ($\epsilon = 0.5$) e outro com uma margem maior ($\epsilon = 1.2$).



2. Regressão SVM

- Reduzir ϵ aumenta o número de vetores de suporte, o que regulariza o modelo. Além disso, se você adicionar mais instâncias de treinamento dentro da margem, isso não afetará as previsões do modelo; por isso, diz-se que o modelo é **ϵ -insensível**. Para realizar regressão SVM linear, você pode usar a classe **LinearSVR** do Scikit-Learn. O código a seguir gera o modelo representado à esquerda na Figura abaixo.

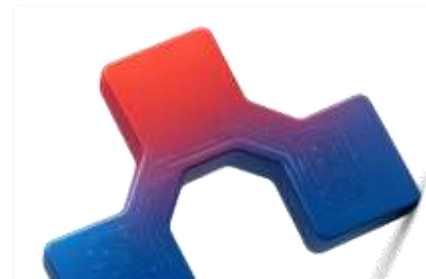


2. Regressão SVM

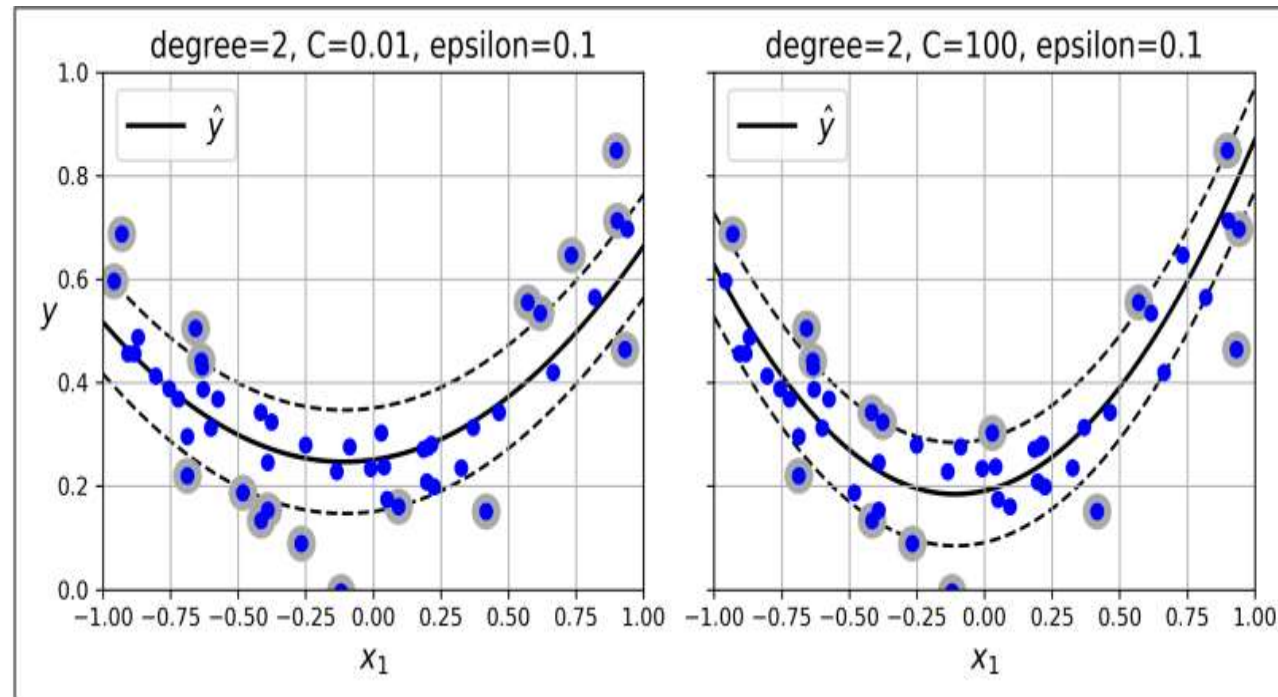
```
from sklearn.svm import LinearSVR

X, y = [...] # a linear dataset
svm_reg = make_pipeline(StandardScaler(),
                        LinearSVR(epsilon=0.5, random_state=42))
svm_reg.fit(X, y)
```

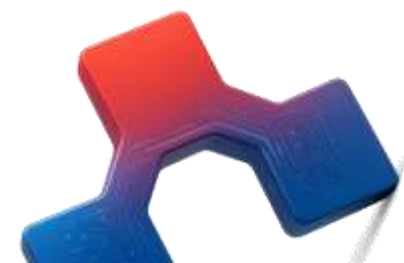
- Para resolver tarefas de regressão não linear, você pode usar um **modelo SVM com kernel**. A Figura a seguir mostra a regressão SVM em um conjunto de treinamento quadrático aleatório, utilizando um kernel polinomial de segundo grau. No gráfico à esquerda há mais regularização (valor de **C** pequeno), enquanto no da direita há menos regularização (valor de **C** grande).



2. Regressão SVM



- O código a seguir utiliza a classe SVR do Scikit-Learn (que oferece suporte ao *kernel trick*) para produzir o modelo representado à esquerda na figura acima.



2. Regressão SVM

```
from sklearn.svm import SVR

X, y = [...] # a quadratic dataset
svm_poly_reg = make_pipeline(StandardScaler(),
                             SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1))

svm_poly_reg.fit(X, y)
```

- A classe SVR é o equivalente para regressão da classe SVC, e a classe LinearSVR é o equivalente para regressão da classe LinearSVC. A classe LinearSVR escala de forma linear com o tamanho do conjunto de treinamento (assim como a classe LinearSVC), enquanto a classe SVR se torna muito lenta quando o conjunto de treinamento cresce demais (assim como a classe SVC).



